# FPGA Implementation of Euclid’s GCD Algorithm

# Problem defination

- The **Greatest Common Divisor (GCD)** of two positive integers is the largest number that divides both integers without leaving a remainder.
- One of the most hardware-friendly methods to compute GCD is **Euclid's subtraction-based algorithm**, which repeatedly subtracts the smaller number from the larger one until both numbers become equal. The final equal value represents the GCD.

# Tasks

- In this experiment, you will implement Euclid's GCD algorithm on an **FPGA Boolean/Basys3 development board** using switches, LEDs, and push buttons. The design must also handle invalid input conditions and provide a visual error indication using a blinking LED.
- If either input number is zero, the GCD is undefined. In such a case, the circuit must raise an error signal, stop computation, and blink an LED at 1 Hz to alert the user. For valid inputs, the algorithm continues until completion and displays the computed GCD on LEDs.

# Learning outcome

- Algorithm-to-hardware mapping
- Finite State Machine (FSM) based control
- Datapath design using registers, comparator, and subtractor
- Clock-based timing (1 Hz LED blinking)
- Interfacing FPGA logic with board-level I/O

# Specification: GCD computation system

## Functional Requirements

1. Two unsigned integers **A** and **B** shall be taken as inputs using slide switches.
2. When the **GO** push button is pressed, the GCD computation shall begin.
3. If **A = 0** or **B = 0**, the system shall:
   - Assert an **error signal**
   - Blink an LED at **1 Hz frequency**
   - Stop further computation
4. If inputs are valid, Euclid's subtraction algorithm shall execute until:
   - `A == B`
   - The final GCD value is stored in register **A**
5. After successful completion:
   - A **DONE** signal shall be asserted
   - The GCD value shall be displayed on LEDs
6. The system shall remain in the final state until **RESET** is pressed.

# Euclid GCD pseudocode

```
START:
    done  = 0
    error = 0

    A = a_in
    B = b_in

    // Error check
    IF (A == 0) OR (B == 0) THEN
        error = 1
        done  = 1
        GOTO HALT
    ENDIF
```

```
LOOP:
    WHILE (A != B) DO
        IF (A > B) THEN
            A = A - B
        ELSE
            B = B - A
        ENDIF
    ENDWHILE

    // Loop terminated => A == B and A holds the GCD
    gcd_out = A
    done = 1
```

# Input–Output Mapping (Board Level)

## Inputs

| Signal | Board Resource |
|---|---|
| Input A | Slide switches (lower half) |
| Input B | Slide switches (upper half) |
| GO | Upper push button |
| RESET | Centre push button |
| Clock | On-board system clock |

## Outputs

| Signal | Board Resource |
| --- | --- |
| GCD Output | 8 LEDs |
| DONE | Extreme left LED |
| ERROR | Extreme left-2 LED (blinks at 1 Hz) |

## LED Behavior Summary

- **DONE LED**
  - Turns ON when GCD computation is complete
- **ERROR LED**
  - Blinks at **1 Hz** when `A = 0` or `B = 0`
  - Remains OFF during normal operation
- **GCD LEDs**
  - Display the final GCD value in binary